

# Modelo de Ising:

## Simulación Numérica y Transición de Fase

Monte Carlo · Metropolis-Hastings · Temperatura Crítica

Prof. Carlos Pinilla

Física Ambiental — Universidad del Valle

May 30, 2026

● Espín +1   ● Espín -1

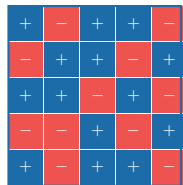
- 1 El Modelo de Ising
- 2 Modelo 1D: caso exactamente soluble
- 3 Hamiltoniano y Distribución de Boltzmann
- 4 Métodos Monte Carlo (MCMC)
- 5 Evolución y Observables
- 6 Temperatura Crítica
- 7 Métodos de Cluster
- 8 Resumen y Notebook

# ¿Qué es el Modelo de Ising?

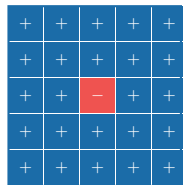
- ▶ Propuesto por **Lenz (1920)**, resuelto por **Ising (1925)**.
- ▶ Modelo de **ferromagnetismo**: partículas con espín binario  $s_i \in \{+1, -1\}$  en una red regular.
- ▶ Los espines **vecinos tienden a alinearse**. La constante  $J$  controla la fuerza.
- ▶ Permite estudiar **transiciones de fase** (orden  $\leftrightarrow$  desorden) al variar  $T$ .
- ▶ Aplicaciones: física estadística, ciencia de materiales, neurociencia, machine learning.

## Variantes

1D (exacto), 2D (exacto — Onsager 1944), 3D (solo numérico)

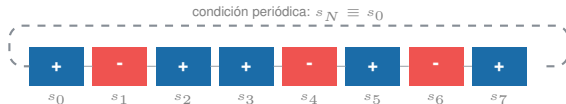


Red 5×5 —  $T$  alta (desordenada)



Red 5×5 —  $T$  baja (ordenada)

# Modelo de Ising 1D — el punto de partida



## Hamiltoniano 1D

$$H = -J \sum_{i=1}^N s_i s_{i+1}, \quad s_{N+1} \equiv s_1$$

Solo interacciones entre **primeros vecinos**.

## Resultado clave

En 1D **no hay**  $T_c$  a  $T > 0$ :  
las fluctuaciones térmicas destruyen el orden  
a cualquier temperatura positiva.

## Solución exacta ( $h = 0$ , c.c. periódica)

$$Z(T) = \lambda_+^N + \lambda_-^N$$

$$\lambda_+ = 2 \cosh\left(\frac{J}{k_B T}\right), \quad \lambda_- = 2 \sinh\left(\frac{J}{k_B T}\right)$$

Para  $N$  grande:  $Z \approx \lambda_+^N$ , y la energía libre:

$$F = -N k_B T \ln \left[ 2 \cosh\left(\frac{J}{k_B T}\right) \right]$$

## Por qué empezar en 1D

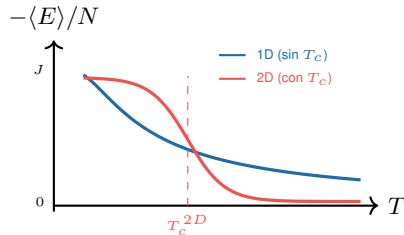
- ▶ Solución analítica → **verificar** la simulación
- ▶ Introduce  $H$ ,  $Z$ ,  $\langle E \rangle$  con álgebra simple
- ▶ Salto al 2D es conceptualmente pequeño

# Modelo 1D — Observables exactos y verificación Monte Carlo

## Energía media exacta ( $h = 0$ , $N$ grande)

$$\langle E \rangle / N = -J \tanh\left(\frac{J}{k_B T}\right)$$

- ▶  $T \rightarrow 0$ :  $\langle E \rangle / N \rightarrow -J$  (todos alineados)
- ▶  $T \rightarrow \infty$ :  $\langle E \rangle / N \rightarrow 0$  (desordenado)
- ▶ Curva **suave**, sin discontinuidades  $\rightarrow$  sin  $T_c$



## Magnetización exacta ( $h = 0$ )

$$\langle |M| \rangle = 0 \quad \forall T > 0$$

No hay magnetización espontánea en 1D. El orden de largo alcance es destruido por fluctuaciones térmicas a cualquier  $T > 0$ .

## Estrategia del notebook

- 1 Simular cadena 1D con Metropolis-Hastings
- 2 Comparar  $\langle E \rangle(T)$  con  $-J \tanh(J/k_B T)$
- 3 Confirmar que no hay  $T_c$  en 1D
- 4 Extender al 2D y observar la transición

# Hamiltoniano y Distribución de Boltzmann

## Energía del sistema (Hamiltoniano 2D)

$$H = -J \sum_{\langle i,j \rangle} (s_{i,j} s_{i+1,j} + s_{i,j} s_{i,j+1})$$

- ▶  $J > 0$ : ferromagnético (vecinos se alinean)
- ▶  $J < 0$ : antiferromagnético
- ▶  $H$  mínima  $\Leftrightarrow$  espines alineados

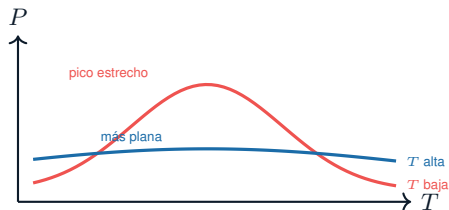
## Función de partición

$$Z(T) = \sum_{\alpha} e^{-H(\alpha)/k_B T}$$

$2^{N^2}$  términos — **intratable** para  $N$  grande.

## Distribución de Boltzmann

$$P(\alpha) = \frac{e^{-H(\alpha)/k_B T}}{Z(T)}$$



## El reto

Calcular  $\langle E \rangle$ ,  $\langle M \rangle$  requiere sumar sobre  $Z$   
 $\Rightarrow$  usamos **Monte Carlo**.

# ¿Por qué Monte Carlo?

## Idea central

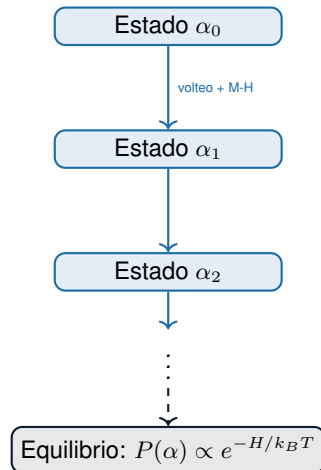
En lugar de sumar *todos* los estados, **muestreamos** configuraciones representativas con probabilidad  $\propto P(\alpha)$ .

$$\langle E \rangle \approx \frac{1}{K} \sum_{k=1}^K E(\alpha_k)$$

## Cadena de Markov (MCMC)

Construimos una cadena en el espacio de estados:

- ▶ Cada nuevo estado depende *solo* del actual.
- ▶ La cadena converge a la **distribución de Boltzmann**.



# Algoritmo Metropolis-Hastings

## Pasos del algoritmo

- 1 **Inicializar** red aleatoria de espines  $\pm 1$ .
- 2 **Proponer**: elegir sitio  $(i, j)$  al azar y voltear su espín.
- 3 **Calcular**  $\Delta E = E_{\text{nuevo}} - E_{\text{actual}}$ .
- 4 **Aceptar/Rechazar**:
  - ▷ Si  $\Delta E \leq 0 \Rightarrow$  aceptar siempre.
  - ▷ Si  $\Delta E > 0 \Rightarrow$  aceptar con probabilidad  $R$ .
- 5 **Repetir** hasta convergencia.

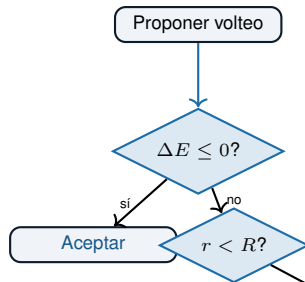
## ¿Por qué funciona?

La cadena satisface el *balance detallado*: la distribución estacionaria es exactamente  $P(\alpha) \propto e^{-H/k_B T}$ .

## Ratio de aceptación

$$R = e^{-\Delta E/k_B T}$$

- ▶  $T$  alta  $\Rightarrow R$  grande  $\Rightarrow$  acepta más
- ▶  $T$  baja  $\Rightarrow R$  pequeño  $\Rightarrow$  más selectivo





## Cambio de energía local

```
1 def delta_energy(lattice, i, j, J=1.0):
2     N = lattice.shape[0]
3     s = lattice[i, j]
4     vecinos = (
5         lattice[(i+1)%N, j] +
6         lattice[(i-1)%N, j] +
7         lattice[i, (j+1)%N] +
8         lattice[i, (j-1)%N]
9     )
10    return 2.0 * J * s * vecinos
```

## Un paso Metropolis-Hastings

```
1 def metropolis_step(lattice, T, J=1.0):
2     N = lattice.shape[0]
3     # Sitio aleatorio
4     i = np.random.randint(0, N)
5     j = np.random.randint(0, N)
6     dE = delta_energy(lattice, i, j, J)
7     # Aceptar / Rechazar
8     if dE <= 0 or \
9         np.random.random() < np.exp(-dE/T):
10        lattice[i, j] += -1
11    return lattice
```

### Truco clave

Con condiciones periódicas,  $\Delta E$  solo depende de los **4 vecinos** — no hay que recalcular toda la energía.

### Un sweep

$N^2$  intentos de volteo = en promedio un intento por espín.

# Variantes MCMC: ¿En qué se diferencian?

| Algoritmo         | ¿Qué<br>voltea?                | se      | Actualización |
|-------------------|--------------------------------|---------|---------------|
| Metropolis-H.     | 1<br>aleatorio                 | espín   | aleatoria     |
| Gibbs             | 1 espín (todos,<br>en orden)   |         | sistemática   |
| Swendsen-<br>Wang | <i>todos</i> los clus-<br>ters |         | global        |
| Wolff             | 1<br>aleatorio                 | cluster | parcial       |



## Metropolis-Hastings

Elige **un sitio aleatorio**, propone voltear; acepta según  $R = e^{-\Delta E/k_B T}$ . Simple y general, pero lento cerca de  $T_c$ .

## Gibbs Sampling

Recorre **todos** los sitios en orden y muestrea cada espín de su distribución condicional exacta. Sin rechazos explícitos; misma eficiencia que M-H.

## S-W & Wolff — métodos de *cluster*

Enlazan espines iguales con  $P=1-e^{-2J/T}$  y voltean **grupos enteros**. Evitan el *critical slowing-down*. SW voltea todos a la vez; Wolff, uno solo.

# Variantes MCMC: Implementación comparada

## Gibbs Sampling (barrido sistemático)

```
1 def gibbs_sweep(lattice, T, J=1.0):
2     N = lattice.shape[0]
3     for i in range(N):
4         for j in range(N):
5             dE = delta_energy(lattice,i,j,J)
6             # Muestrea de la condicional
              exacta
7             if dE <= 0 or \
8                 np.random.random() < np.exp(-
                    dE/T):
9                 lattice[i, j] *= -1
10    return lattice # un sweep completo
```

## Swendsen-Wang (todos los clusters)

```
1 def swendsen_wang_step(lattice, T, J=1.0):
2     clusters = find_clusters(lattice, T, J)
3     for cluster in clusters:
4         if np.random.random() < 0.5:
5             for (i,j) in cluster:
6                 lattice[i,j] *= -1
7     return lattice
```

## Wolff (un cluster aleatorio)

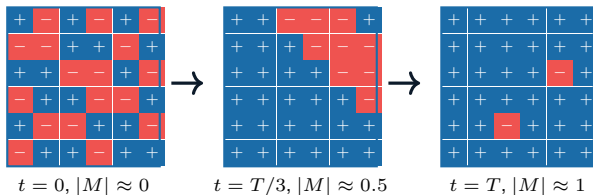
```
1 def wolff_step(lattice, T, J=1.0):
2     N = lattice.shape[0]
3     # Punto de partida aleatorio
4     i0 = np.random.randint(0, N)
5     j0 = np.random.randint(0, N)
6     cluster = grow_cluster(
7         lattice, i0, j0, T, J)
8     # Voltear todo el cluster
9     for (i,j) in cluster:
10        lattice[i,j] *= -1
11    return lattice
```

## Regla de enlace (SW y Wolff)

$$P(\text{enlace} \mid \sigma_i = \sigma_j) = 1 - e^{-2J/T}$$

Si  $\sigma_i \neq \sigma_j$ :  $P = 0$  siempre.

# Evolución de la Red 2D



## Magnetización $M$

$$M = \frac{1}{N^2} \sum_{i,j} s_{i,j}$$

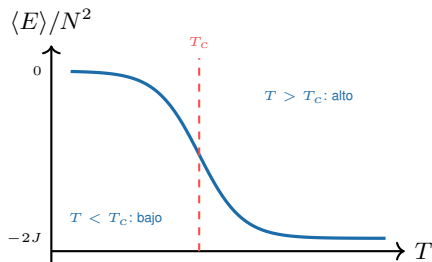
Mide el grado de **alineamiento global**.

## Energía por espín $\langle E \rangle / N^2$

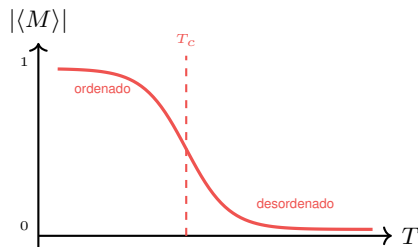
$$2D: -2J \leq \langle E \rangle / N^2 \leq 0$$

Mínima cuando todos los espines están alineados.

# Energía y Magnetización vs. Temperatura



La energía sube abruptamente en  $T_c$  (inflexión sigmoideal).



La magnetización cae abruptamente en  $T_c \rightarrow$  transición de fase.

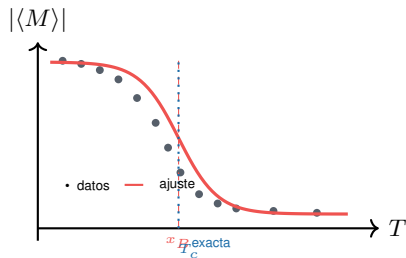
$$\langle E \rangle \approx \frac{1}{K} \sum_k E(\alpha_k) \quad |\langle M \rangle| \approx \frac{1}{K} \sum_k |M(\alpha_k)| \quad (\text{promedios sobre la cadena de Markov})$$

# Temperatura Crítica $T_c$

## Ajuste logístico sobre $|\langle M \rangle|(T)$

$$f(T) = \frac{L}{1 + e^{-k(T-x_B)}} + a$$

Minimizamos RMSE respecto a  $L, k, x_B, a$ .  
El **punto de inflexión**  $x_B$  aproxima  $T_c$ .



## Resultados (2D y 3D)

| Algoritmo     | 2D           | 3D                      |
|---------------|--------------|-------------------------|
| Metropolis-H. | 2.57         | 4.33                    |
| Gibbs         | 2.62         | 4.32                    |
| Swendsen-Wang | 2.34         | 4.27                    |
| Wolff         | 2.39         | 4.35                    |
| <b>Exacto</b> | <b>2.269</b> | <b>4.51<sup>†</sup></b> |

<sup>†</sup> valor numérico de referencia

## Error relativo típico

$$\varepsilon = \frac{|T_c - x_B|}{T_c} \approx 3-5\%$$

# Calor Específico y Susceptibilidad

## Calor específico $C_V$

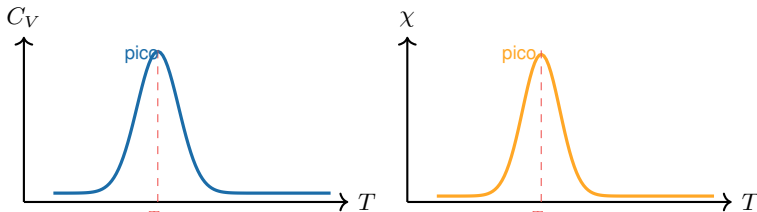
$$C_V = \frac{\langle E^2 \rangle - \langle E \rangle^2}{N^2 T^2}$$

- Mide **fluctuaciones** de energía.
- Presenta un **pico** en  $T_c$ .
- En 2D el pico diverge logarítmicamente (solución exacta de Onsager).

## Susceptibilidad magnética $\chi$

$$\chi = \frac{N^2}{T} (\langle M^2 \rangle - \langle |M| \rangle^2)$$

- Mide **fluctuaciones** de magnetización.
- También diverge en  $T_c$ .
- Ambas divergencias son señales de la transición de fase.



# Más Allá de Metropolis-Hastings: Métodos de Cluster

## Problema: *Critical Slowing-Down*

Cerca de  $T_c$ , Metropolis-Hastings converge **muy lento** — los dominios magnéticos son grandes y un volteo individual apenas los modifica.

### Swendsen-Wang (1987)

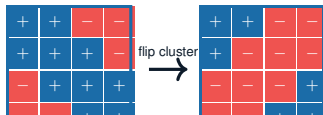
- 1 Formar enlaces entre espines iguales adyacentes con  $P(\text{enlace}) = 1 - e^{-2J/T}$ .
- 2 Identificar *todos* los clusters.
- 3 Voltear cada cluster completo con prob.  $\frac{1}{2}$ .

**Muy rápido** (converge en  $\sim 7$  iteraciones).

### Wolff (1989)

- 1 Misma regla de enlace.
- 2 Elegir *un* espín al azar y crecer su cluster.
- 3 Voltear *solo* ese cluster.

Menor coste por iteración; útil para redes grandes. Más lento en convergencia que SW.



Voltear un cluster entero  $\Rightarrow$  cambio global grande



## Lo que vimos hoy

- ▶ Modelo de Ising: espines  $\pm 1$  en red, Hamiltoniano  $H = -J \sum s_i s_j$ .
- ▶ Distribución de Boltzmann:  $P \propto e^{-H/k_B T}$ .
- ▶ MCMC — Metropolis-Hastings: volteo + aceptar/rechazar.
- ▶ Observables:  $\langle E \rangle(T)$ ,  $|\langle M \rangle|(T)$ ,  $C_V$ ,  $\chi$ .
- ▶ Temperatura crítica  $T_c \approx 2.27$  (2D, exacto).
- ▶ Métodos de cluster: Swendsen-Wang, Wolff.

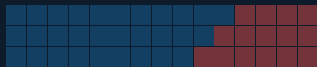
## Agenda del Notebook

- 1 Implementar `metropolis_step` paso a paso.
- 2 Visualizar la evolución de la red 2D.
- 3 Barrer temperaturas  $T \in [1, 4.5]$  y graficar  $\langle E \rangle(T)$  y  $|\langle M \rangle|(T)$ .
- 4 Estimar  $T_c$  con ajuste logístico.
  - + Ej 1: Efecto del tamaño  $N$ .
  - + Ej 2: Inicio frío vs. caliente.
  - + Ej 3: Calcular  $\chi(T)$ .

## Librerías

`numpy`, `matplotlib`, `scipy` (estándar

Appendix)



# ¡A simular!

Abran el notebook `Ising_Notebook.ipynb`

●  $s_i = +1$       ●  $s_i = -1$        $R = e^{-\Delta E/k_B T}$